

PROJECT REPORT
ON
“CAR PARKING SYSTEM USING CCTV CAMERA’S”

SUBMITTED BY
GAURAV PATIL
SHUBHAM PATIL
VRUSHAB SAWANT

UNDER THE GUIDENCE OF
JAGAT GAYDHANE



DEPARTMENT OF INFORMATION TECHNOLOGY
DATTA MEGHE COLLEGE OF ENGINEERING
SECTOR -3, AIROLI, NAVI MUMBAI- 400 708, (M.S.), INDIA
2019-2020

PROJECT REPORT
ON
“CAR PARKING SYSTEM USING CCTV CAMERA’S”

SUBMITTED TO THE
UNIVERSITY OF MUMBAI, MUMBAI

Proposed to be submitted in the partial fulfillment of requirement for the
Degree of Bachelor of Engineering in Information Technology

Submitted by

GAURAV PATIL B/36

SHUBHAM PATIL B/37

VRUSHAB SAWANT B/48

UNDER THE GUIDENCE OF
JAGAT GAYDHANE



DEPARTMENT OF INFORMATION TECHNOLOGY
DATTA MEGHE COLLEGE OF ENGINEERING
SECTOR -3, AIROLI, NAVI MUMBAI- 400 708, (M.S.), INDIA

2019-2020

Datta Meghe College of Engineering

(AICTE & Govt. of Maharashtra Recognized, Affiliated to University of Mumbai)

Department of Information Technology



C E R T I F I C A T E

Date:

This is to certify that, the project work embodied in this report entitled, “**CAR PARKING SYSTEM USING CCTV CAMERA’S**” submitted by Gaurav Patil, Shubham Patil, Vrushab Sawant for the award of **Bachelor of Engineering (B.E.)** degree in the subject of **Information Technology**, is a work carried out by them under my guidance and supervision within the institute. The work described in this project report is carried out by the concerned student/s and has not been submitted for the award of any other degree of the University of Mumbai.

Further, it is certify that the student/s was regular during the academic year and has worked under the guidance of concerned faculty until the submission of this project work at the Datta Meghe College of Engineering.

Signature of the Guide

Signature of Head of Department

Signature of Principal



DATTA MEGHE COLLEGE OF ENGINEERING

Plot no 98, Cidco, Sector-3 Post Box No.15 Airoli, Navi Mumbai-400708

CERTIFICATE OF APPROVAL

**Project Entitled: “CAR PARKING SYSTEM USING CCTV
CAMERA’S”**

SUBMITTED BY

GAURAV PATIL

SHUBHAM PATIL

VRUSHAB SAWANT

**In the partial fulfillment of the degree of B.E in “INFORMATION
TECHNOLOGY” is approved.**

Signature of Internal Examiner

Signature of External Examiner

Signature of Head of Department

Signature of Principal

DECLARATION BY THE CANDIDATE

We wish to declare that the work embodied in this project report entitled “CAR PARKING SYSTEM USING CCTV CAMERA’S” forms our own contribution to the project work carried out under the guidance of Mr. Jagat Gaydhane Asst. Prof., Department Of Information Technology, Datta Meghe College Of Engineering, Airoli, Navi Mumbai, affiliated to University Of Mumbai.

This work is carried out, written, compiled and submitted by us for the award of degree of Bachelor of Engineering in Information Technology at the University of Mumbai. This work has not been submitted for any other degree of this University or any other University.

Mumbai

Date:

Signature of candidate/s

Acknowledgement

We express our sincere gratitude towards our guide Mr. Jagat Gaydhane for her constant help, encouragement and inspiration till date the project work. Without her invaluable guidance this work would never have been a reached to this level. We also like to thank HOD Dr. Sujata kolhe and all the faculty members of our IT department for their support, feedback and help throughout the project.

Thanking you:

Gaurav Patil - IT/B - 36

Shubham Patil - IT/B – 37

Vrushab Sawant – IT/B - 48

ABSTRACT

Parking is a problem for almost everyone today so there has to be a solution, which helps getting rid of problems arising due to the lack of a proper parking management system. With the development of automobile industry, the number of private cars is greatly increasing. Management of parking has grown to large extent. Parking the car today need parking policies for safety and security reasons. There is always competition for the parking space. A good solution to overcome parking crises would be by increasing the number of parking spaces or else enlarge the parking lots, but this will lead to huge investment. However better management of existing parking spaces will be wise method. The availability of parking spaces should be improved by informing user about available parking space and guiding him accordingly.

TABLE OF CONTENTS

CHAPTER NO.	TITLE OF CHAPTER	PAGE NUMBER
	Acknowledgement	6
	Abstract	7
	List of Figures	10
	List of Tables	10
CHAPTER 1 1.1 1.2 1.3 1.4 1.5 1.6 1.7	<u>INTRODUCTION</u> Overview Background Problem Statement Objective and Purpose Period of the project Scope of the project Project Specifications	11-16
CHAPTER 2	<u>LITERATURE REVIEW</u>	17-19
CHPTER 3 3.1 3.2 3.3 3.4 3.5	<u>METHODOLOGY</u> Tools and Technologies DFD Sequence Diagram Use Case Diagram Component Diagram	20-30

CHAPTER 4 4.1	<u>IMPLEMENTATION</u> Coding	31-34
CHAPTER 5 5.1 5.2	<u>RESULTS AND DISCUSSION</u> Data Collection (DATABASE) Screen Shots	35-39
CHAPTER 6	<u>CONCLUSION</u>	40-41
CHAPTER 7	<u>FUTURE SCOPE</u>	42-43
	<u>REFERENCES</u>	44

List of Figure

Fig No.	Figure Name	Page No.
3.2	DFD	26-27
3.3	SEQUENCE DIAGRAM	28
3.4	USE CASE DIAGRAM	29
3.5	COMPONENT DIAGRAM	30

List of Tables

Table No.	Table Name	Page No.
1.5	Period of Project	15

CHAPTER 1

INTRODUCTION

INTRODUCTION

Presently, the detection technique of laser, radar, infrared ray and ultrasonic have been widely applied at the aspects of detection of car for parking lot. At the aspect of distance measurement, the technique of ultrasonic is applied to measure the detection range when a car is parked to detect the availability of the parking lot.

The Smart System of Ultrasonic Car Parking introduced in this thesis can automatically measure the distance between the trail of the car and parking lot and detect the car is parked or not, further more it can show the available parking lot and user can book the available parking lot in real time, so it can ensure the car parking and reduced the parking overhead. With this system, the user can know either he is getting parking lot or not. This system will have a prosperous application prospect.

Now a day there is growing popularity and affordability of internet –enabled smartphones and because of data available online we can step to solve parking problem. Android smartphone enables user to virtually carry the internet with him. By using this system the user can determine the availability of parking spaces prior to entering the parking lot. The system provides a visual display to the user regarding the available parking spaces by which he can book or reserve a space whenever he needs.

1.1. Overview

A car parking system is a mechanical device that multiplies parking capacity inside a parking lot. Parking systems are generally powered by electric motor or hydraulic pumps that move vehicles into a storage position.

Car parking systems may be traditional or automated. Automatic multi-story automated car park systems are less expensive per parking slot, since they tend to require less building volume and less ground area than a conventional facility with the same capacity. In the long term, automated car parking systems are likely to be more cost effective than traditional parking garages. Car parking systems reduce exhaust gas cars need not drive around in search of street parking spaces.

1.2. Background

Over the years, car parking systems and the accompanying technologies have increased and diversified. Car parking systems have been around almost since the time cars were invented. In any area where there is a significant amount of traffic, there are car parking systems. Car Parking systems were developed in the early 20th century in response to the need for storage space for vehicles.

1.3. Problem Statement

System Setup The host car is equipped with an array of 12 ultrasonic sensors. The single sensors are placed equally on its front and rear. Within this work only six of them are used, three on each side of the car: the front-side and the rear sensor (aperture 75°) and additionally the passive rear-side sensor (aperture 50°), which only receives ultrasonic echoes emitted by the rear sensor. All sensors are operating at a rate of 30ms. The region of special interest for lane-change maneuvers and in which objects shall be tracked is depicted in figure 1. It reaches (back to front) from 3 m behind the car up to the side mirrors and covers roughly three to four meters to the side. Ideally, a possible track should converge within 0.3 s after a car has entered this region. The system should operate under different weather conditions and in diverse environments (city, rural roads, autobahn). However, in the following we will focus on high-speed scenarios on the autobahn under mild weather conditions. This means heavy rain or snow will not be taken into account. Typically, vehicle detection and tracking are separated in two individual steps using high-resolution sensor data like laser-range-scans, to first detect an object and then feed this estimate to a tracking filter. Due to the very limited information that can be gathered from us-sensors, our approach dispenses

with data segmentation, data association or separated detection. In contrast, the data from all sensors is merged immediately into one common state-space. As there might be more than one object – it is usually one or two – in the region of interest the resulting probability density function to represent the track might take the form of a multimodal distribution. Therefore, a particle filter with some mixture tracking capabilities was chosen to implement the tracking algorithm.

1.4. Objective and Purpose

Objectives of Proposed Design Proposed parking system would save time and provide comfortable hazard free parking experience to the users. Features of the parking management system are as listed below: Monitoring of parking space and updated indication of vacant parking slots.

- Assistance to the parking place via displays.
- Safe parking assistance using ultrasonic sensors.
- Automatic record of check-in and check-outs of the cars/vehicles
- payment of parking charges from the user's account.

1.5. Period of the Project

The following table gives the project plan for the Phase 1 & 2 of our project:

Activity	Description	Effort in person weeks	Deliverable
Phase 1			
P1-01	Requirement Analysis	2 weeks	Requirement Gathering
P1-02	Existing System Study & Literature	3 weeks	Existing System Study & Literature
P1-03	Technology Selection	2 weeks	>NET
P1-04	Modular Specifications	2 weeks	Module Description
P1-05	Design & Modeling	4 weeks	Analysis Report
	Total	13 weeks	

Activity	Description	Effort in person weeks	Deliverable
Phase 2			
P2-01	Detailed Design	2 weeks	LLD / DLD Document
P2-02	UI and user interactions design	Included in above	UI document
P2-03	Coding & Implementation	12 weeks	Code Release
P2-04	Testing & Bug fixing	2 weeks	Test Report
P2-05	Performance Evaluation	4 weeks	Analysis Report
P2-06	Release	Included in above	System Release
	Total	20 weeks	Deployment efforts are extra

1.6. Scope of the Project

This project aims to design a smart parking system that is sensor free and functions as an indicator of the vacant spaces to the oncoming users of the parking lot. Parking control system has been generated in such a way that it is filled with many secure devices such as barricades, swing gates, slide gates, parking control gates, toll gates, time and attendance machine, car counting system etc.

1.7. Project Specifications

- **USER MODULE (WEB APP):**
 - Users can Register themselves at the Application
 - User can Login into the Application
 - Search Available area for parking.
 - Book slot for parking space

CHAPTER 2

LITERATURE REVIEW

LITERATURE REVIEW

State-of-the-art Parking Management

Traffic searching for parking comparison under different parking guidance strategies. Many parking guidance systems have been developed over the past decade. In this subsection, we study several existing parking guidance approaches and explain their limitations. Furthermore, we simulate these different parking management strategies under realistic traffic and parking conditions, compare their performance, and show results.

Blind Search

Blind searching is the simple strategy applied by users when there is no parking information. In this case, the drivers keep cruising for parking spaces within a certain distance to their destination. The drivers will stop searching until finding any available space. Otherwise, the drivers will extend the searching area and continuously look for vacant spaces in the neighboring parking lots.

Parking Information Sharing (PIS)

This mechanism is commonly adopted by the current state of the smart Parking system design. After the smart parking system publishes the parking availability information to the drivers in certain area, the driver will decide their desired parking destination where the parking lot has available spaces, according to the obtained parking availability information. However, if the number of vacant spaces in a parking lot is very limited in busy hours, it is likely that the number of drivers in demand for these parking spaces, which is based on parking information. This phenomenon is called “multiple-car-chasing-single space”, which may cause severe congestion.

Buffered PIS (BPIS)

To address the problematic “multiple-car-chase-single slot” phenomenon, some designers of smart parking systems modify the PIS mechanism. They intentionally reduce the number of vacant spaces, when publishing the live availability information to keep a buffer. Therefore, though there may be more drivers pursuing the limited available spaces, the system has some extra spaces to avoid the conflict. But it is difficult to determine the number of the buffer spaces. If the buffer is too small, the problem of “multiple-car-chase-single-space” will not be eliminated. If it is too large, the utilization of parking spaces will be low. As alluded to above, the blind search system is an open loop system, where users make decision without looking at the state of the system. The PIS

and BPIS strategies allow drivers to make decisions based on the system state (e.g., parking availability information). However, the phenomena of multiple car-chase-single-spaces cannot be fully eliminated. To reduce the traffic searching for parking, we suggest a reservation based system, where drivers make reservations through the parking management system. If a driver makes the reservation successfully, it guarantees an available parking space for him, and the driver can park at the reserved space without searching. The reservation-based system allows drivers to select the most convenient parking space under their budget constraints.

CHAPTER 3

METHODOLOGY

3.1. HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS:

- 4 GB RAM.
- 256 GB HDD.
- Intel 1.8 GHz i3 Processor
- Microcontroller
- I.R sensor

SOFTWARE REQUIREMENTS:

- Windows XP, Windows 7,8
- Visual Studio 2013
- Microsoft SQL server 2012
- Windows Operating System

SOFTWARES USED FOR THE PROJECT

1. ASP.NET

ASP.NET is more than the next version of Active Server Pages (ASP); it is a unified Web development platform that provides the services necessary for developers to build enterprise-class Web applications. While ASP.NET is largely syntax-compatible with ASP, it also provides a new programming model and infrastructure that enables a powerful new class of applications. You can migrate your existing ASP applications by incrementally adding ASP.NET functionality to them. ASP.NET is a compiled .NET Framework -based environment. You can author applications in any .NET Framework compatible language, including Visual Basic and Visual C#. Additionally, the entire .NET Framework platform is available to any ASP.NET application. Developers can easily access the benefits of the .NET Framework, which include a fully managed, protected, and

feature-rich application execution environment, simplified development and deployment, and seamless integration with a wide variety of languages.

.net Framework:

The .NET Framework is Microsoft's Managed Code programming model for building applications on Windows clients, servers, and mobile or embedded devices. Microsoft's .NET Framework is a software technology that is available with several Microsoft Windows operating systems. In the following sections describes, the basics of Microsoft .Net Frame work Technology and its related programming models.

C# is a language for professional programming. C# (pronounced C sharp) is a programming language designed for building a wide range of enterprise applications that run on the .NET Framework. The goal of C# is to provide a simple, safe, modern, object-oriented, high-performance, robust and durable language for .NET development. Also, it enables developers to build solutions for the broadest range of clients, including Web applications, Microsoft Windows Forms-based applications, and thin- and smart-client devices.

2. Microsoft SQL Server

Relational database management:

A relational database management system uses only its relational capabilities to manage the information stored in its database.

Information Representation:

All information stored in a RDBMS is represented only by data item values, which are stored in the tables that makeup the database.

Logical Accessibility:

Every data item value stored in a relational database is accessible by stating the name of the table it is stored in, the name of the column under which it is stored and the value of the primary key that defines the row in which it is stored.

Representation of Null Values:

The database management system has a consistent method for representing null values. For example, null values for numeric data must be distinct from zero or any other numeric value and for character data it must be different from a string of blanks or any other character value

Catalog Facility:

The logical description of the relational database management system is represented in the same manner as ordinary data. This is done so that the facilities of the relational database management system itself can be used to maintain database description.

Data Language:

A relational database management system may support many types of languages for description data and accessing the database. However, there must be at least one language that uses ordinary character strings to support the definition of data, the definition of views, the manipulation of data, constraints on data integrity, information concerning authorization and the boundaries for recovery of units.

View Updating:

Any view that can be defined using combinations of base tables, that are theoretically updateable, is capable of being updated by the relational database management system.

Insert, Update and Delete:

Any operand that describes the results of a single retrieval operation is capable of being applied to an insert, update or delete operation as well.

Physical Data Independence:

Changes made to physical storage representations or access methods do not require changes to be made to application programs.

Logical Data Independence:

Changes made to tables, that do not modify any data stored in that table, do not require changes to be made to application programs.

Integrity Constraints:

Constraints that apply to entity integrity and referential integrity are specifiable by the data language implemented by the database management system and not by the statements coded into the application programs.

Database Distribution:

The data language implemented by the relational database management system supports the ability to distribute the database without requiring changes to be made application programs.

Not Subversion:

If the relational database management supports facilities that allow application programs to operate on the table's a row at a time, an application program using this type of database access is prevented from bypassing entity integrity or referential integrity constraints that are defined for the database. Business today demands a different kind of data management solution. Performance scalability, and reliability are essential, but businesses now expect more from their key IT investment. SQL Server 2005 exceeds dependability requirements and provides innovative capabilities that increase employee effectiveness, integrate heterogeneous IT ecosystems, and maximize capital and operating budgets. SQL Server 2005 provides the enterprise data management platform your organization needs to adapt quickly in a fast-changing environment. Benchmarked for scalability, speed, and performance, SQL Server 2005 is a fully enterprise-class database product, providing core support for Extensible Markup Language (XML) and Internet queries.

Easy-to-use Business Intelligence(BI) Tools:

Through rich data analysis and data mining capabilities that integrate with familiar applications such as Microsoft Office, SQL Server 2005 enables you to provide all of your employees with critical, timely business information tailored to their specific information needs. Every copy of SQL Server 2005 ships with a suite of BI services

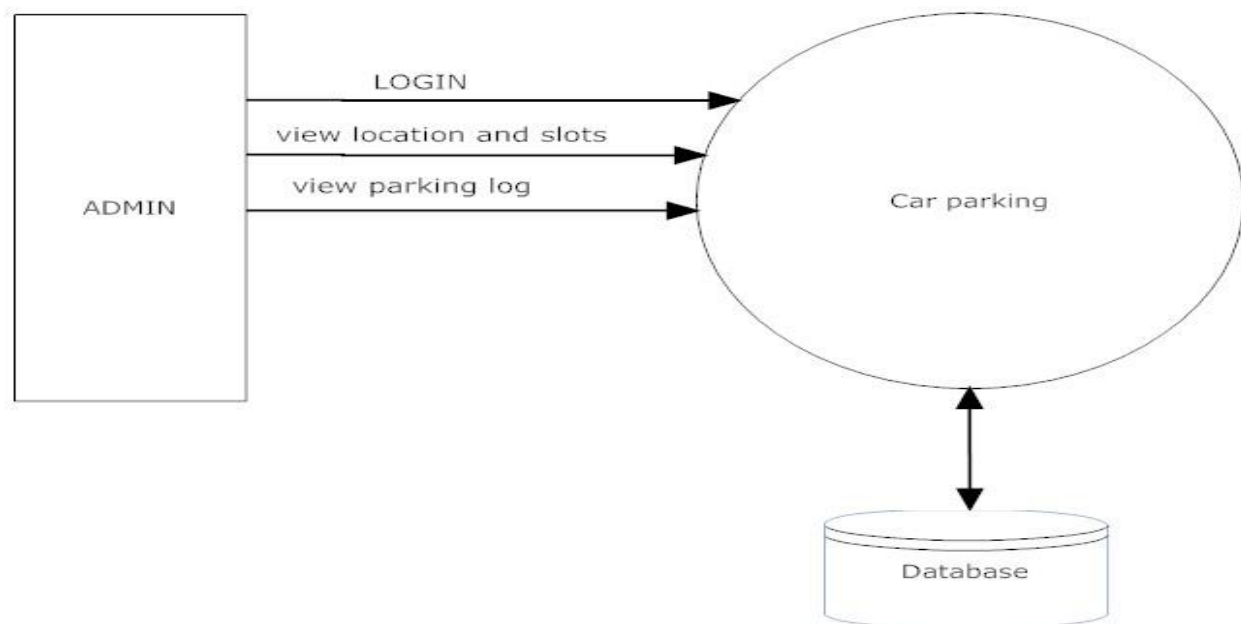
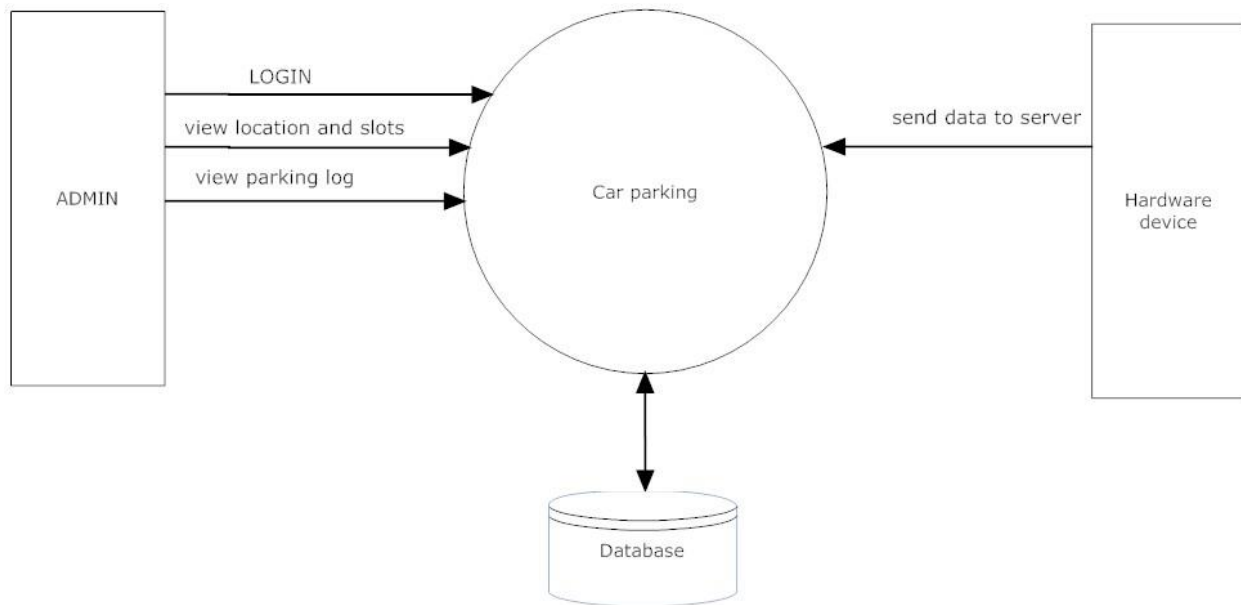
Self-Tuning and Management Capabilities:

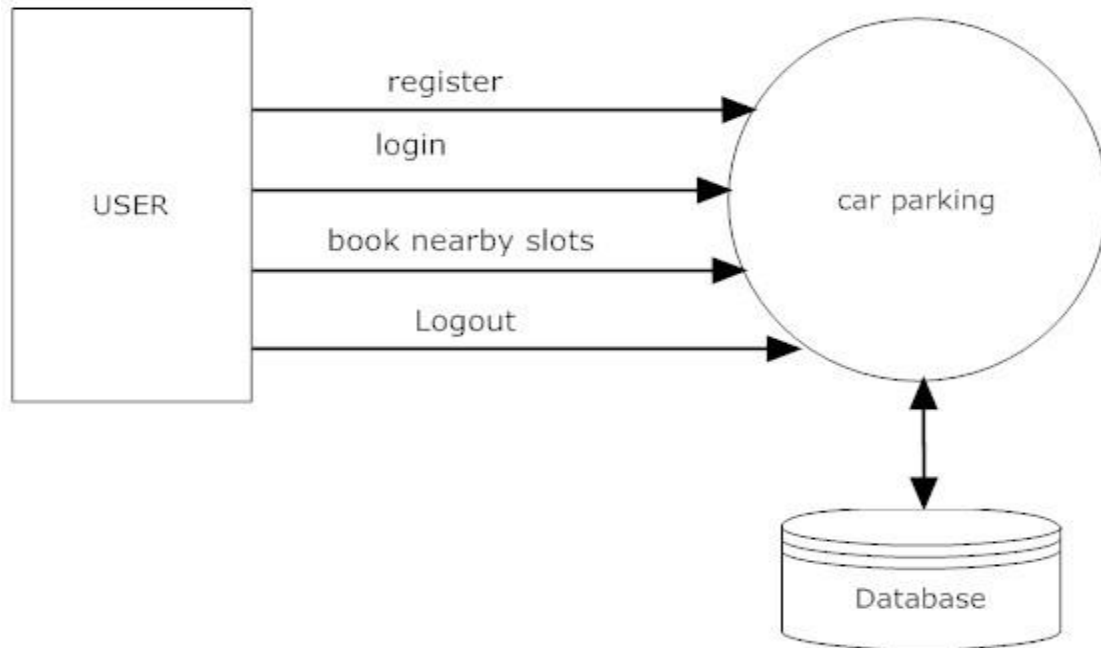
Revolutionary self-tuning and dynamic self-configuring features optimize database performance, while management tools automate standard activities. Graphical tools and performance, wizards simplify setup, database design, and performance monitoring, allowing database administrators to focus on meeting strategic business needs.

Data Management Application and Services:

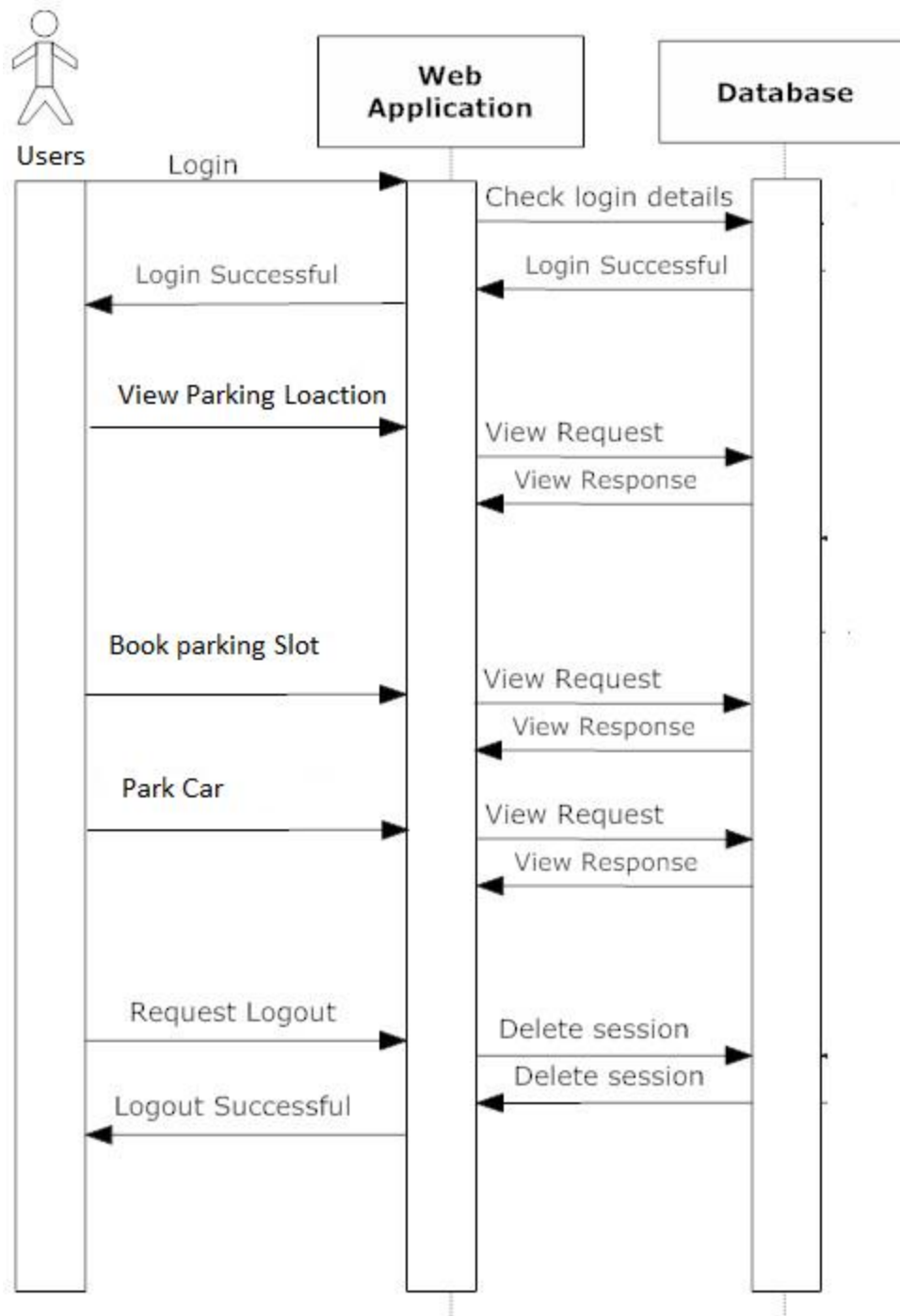
Unlike its competitors, SQL Server 2005 provides a powerful and comprehensive data management platform. Every software license includes extensive management and development tools, a powerful extraction, transformation, and loading (ETL) tool, business intelligence and analysis services, and analysis service, and such as Notification Service. The result is the best overall business value available.

3.2. DFD DIAGRAM

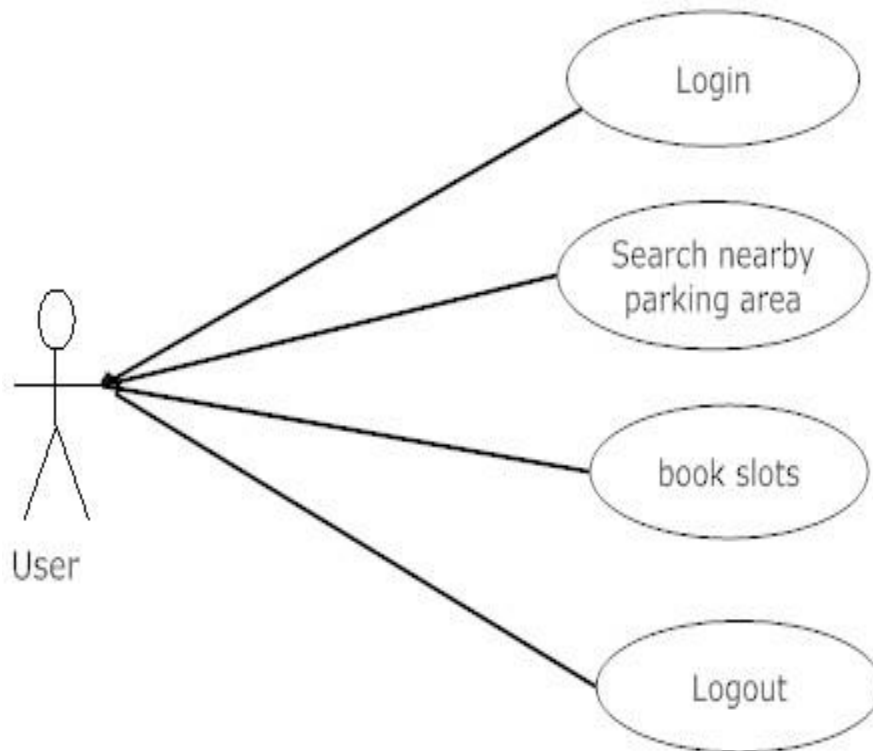




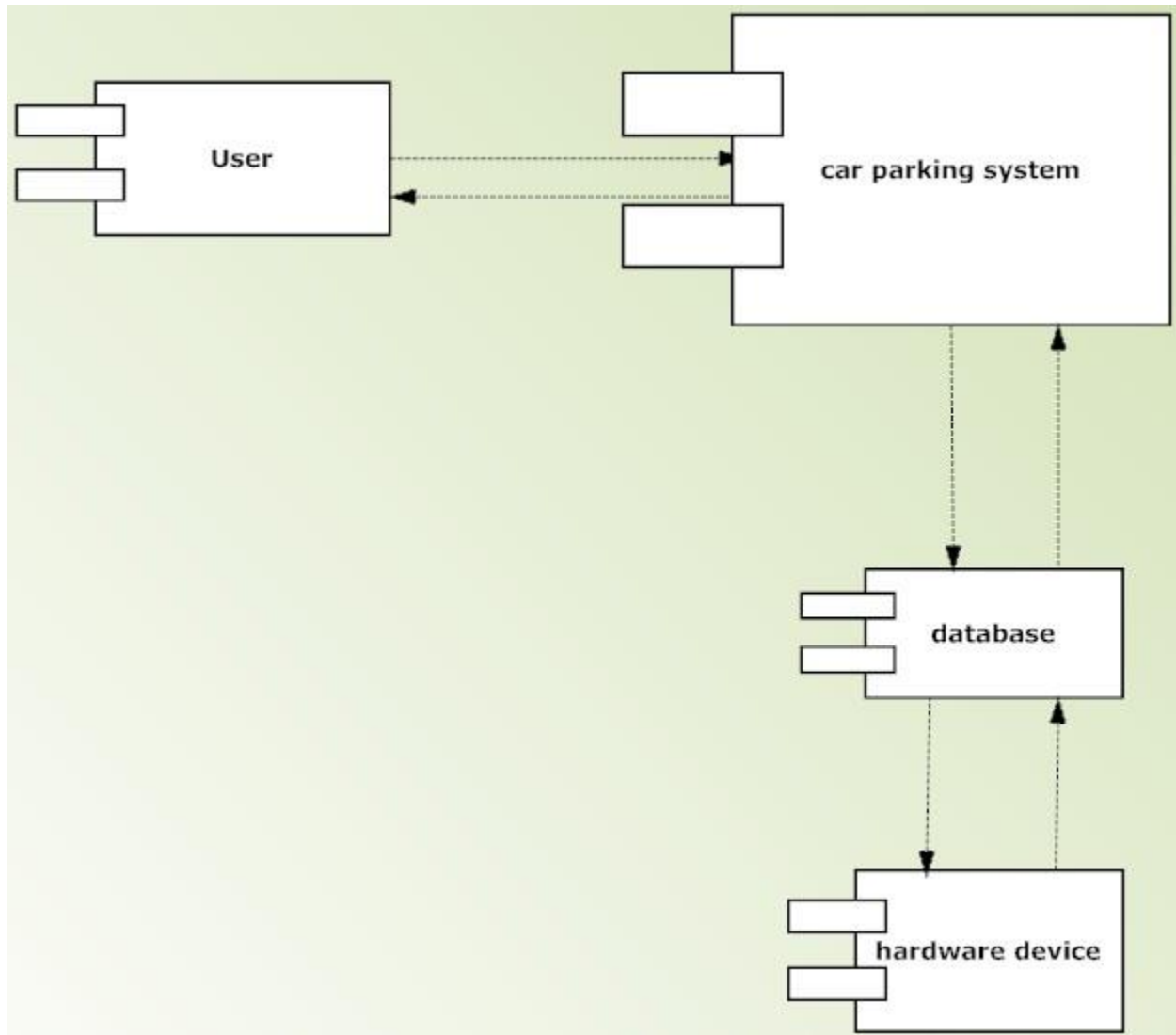
3.3. SEQUENCE DIAGRAM



3.4. USE CASE DIAGRAM



3.5. COMPONENT DIAGRAM



CHAPTER 4

IMPLEMENTATION

4.1 CODING

```
apnr.py - C:\Users\Admin\Desktop\Final_Project\apnr.py (3.6.3)
File Edit Format Run Options Window Help

from skimage.segmentation import clear_border
import pytesseract
import numpy as np
import imutils
import cv2

class ANPR:
    def __init__(self, minAR=4, maxAR=5, debug=False):
        # store the minimum and maximum rectangular aspect ratio
        # values along with whether or not we are in debug mode
        self.minAR = minAR
        self.maxAR = maxAR
        self.debug = debug

    def debug_imshow(self, title, image, waitKey=False):
        # check to see if we are in debug mode, and if so, show the
        # image with the supplied title
        if self.debug:
            cv2.imshow(title, image)
            # check to see if we should wait for a keypress
            if waitKey:
                cv2.waitKey(0)

    def locate_license_plate_candidates(self, gray, keep=5):
        # perform a blackhat morphological operation that will allow
        # us to reveal dark regions (i.e., text) on light backgrounds
        # (i.e., the license plate itself)
        rectKern = cv2.getStructuringElement(cv2.MORPH_RECT, (13, 5))
        blackhat = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT, rectKern)
        self.debug_imshow("Blackhat", blackhat)

        # next, find regions in the image that are light
        squareKern = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
        light = cv2.morphologyEx(gray, cv2.MORPH_CLOSE, squareKern)
        light = cv2.threshold(light, 0, 255,
                              cv2.THRESH_BINARY | cv2.THRESH_OTSU)[1]
        self.debug_imshow("Light Regions", light)

        # compute the Scharr gradient representation of the blackhat
        # image in the x-direction and then scale the result back to
        # the range [0, 255]
        gradX = cv2.Sobel(blackhat, cv2.CV_32F, 1, 0,
                          kernelSize=3, scale=100,
                          borderType=cv2.BORDER_REPLICATE)
        gradX = 255 * ((gradX - minVal) / (maxVal - minVal))
        gradX = gradX.astype("uint8")
        self.debug_imshow("Scharr", gradX)

        # blur the gradient representation, applying a closing
        # operation, and threshold the image using Otsu's method
        gradX = cv2.GaussianBlur(gradX, (5, 5), 0)
        gradX = cv2.morphologyEx(gradX, cv2.MORPH_CLOSE, rectKern)
        thresh = cv2.threshold(gradX, 0, 255,
                              cv2.THRESH_BINARY | cv2.THRESH_OTSU)[1]
        self.debug_imshow("Grad Threshold", thresh)

        # perform a series of erosions and dilations to clean up the
        # thresholded image
        thresh = cv2.erode(thresh, None, iterations=2)
        thresh = cv2.dilate(thresh, None, iterations=2)
        self.debug_imshow("Grad Erode/Dilate", thresh)

        # take the bitwise AND between the threshold result and the
        # light regions of the image
        thresh = cv2.bitwise_and(thresh, thresh, mask=light)
        thresh = cv2.dilate(thresh, None, iterations=2)
        thresh = cv2.erode(thresh, None, iterations=1)
        self.debug_imshow("Final", thresh, waitKey=True)

        # find contours in the thresholded image and sort them by
        # their size in descending order, keeping only the largest
        # ones
        cnts = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL,
                              cv2.CHAIN_APPROX_SIMPLE)
        cnts = imutils.grab_contours(cnts)
        cnts = sorted(cnts, key=cv2.contourArea, reverse=True)[:keep]
        # return the list of contours
        return cnts

    def locate_license_plate(self, gray, candidates,
                             clearBorder=False):
        # initialize the license plate contour and ROI
        lpCnt = None
        roi = None
        # loop over the license plate candidate contours
```

```
apnr.py - C:\Users\Admin\Desktop\Final_Project\apnr.py (3.6.3)
File Edit Format Run Options Window Help

        gradX = 255 * ((gradX - minVal) / (maxVal - minVal))
        gradX = gradX.astype("uint8")
        self.debug_imshow("Scharr", gradX)

        # blur the gradient representation, applying a closing
        # operation, and threshold the image using Otsu's method
        gradX = cv2.GaussianBlur(gradX, (5, 5), 0)
        gradX = cv2.morphologyEx(gradX, cv2.MORPH_CLOSE, rectKern)
        thresh = cv2.threshold(gradX, 0, 255,
                              cv2.THRESH_BINARY | cv2.THRESH_OTSU)[1]
        self.debug_imshow("Grad Threshold", thresh)

        # perform a series of erosions and dilations to clean up the
        # thresholded image
        thresh = cv2.erode(thresh, None, iterations=2)
        thresh = cv2.dilate(thresh, None, iterations=2)
        self.debug_imshow("Grad Erode/Dilate", thresh)

        # take the bitwise AND between the threshold result and the
        # light regions of the image
        thresh = cv2.bitwise_and(thresh, thresh, mask=light)
        thresh = cv2.dilate(thresh, None, iterations=2)
        thresh = cv2.erode(thresh, None, iterations=1)
        self.debug_imshow("Final", thresh, waitKey=True)

        # find contours in the thresholded image and sort them by
        # their size in descending order, keeping only the largest
        # ones
        cnts = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL,
                              cv2.CHAIN_APPROX_SIMPLE)
        cnts = imutils.grab_contours(cnts)
        cnts = sorted(cnts, key=cv2.contourArea, reverse=True)[:keep]
        # return the list of contours
        return cnts

    def locate_license_plate(self, gray, candidates,
                             clearBorder=False):
        # initialize the license plate contour and ROI
        lpCnt = None
        roi = None
        # loop over the license plate candidate contours
```

```
apnr.py - C:\Users\Admin\Desktop\Final_Project\apnr.py (3.6.3)
File Edit Format Run Options Window Help

    # display any debugging information and then break
    # from the loop early since we have found the license
    # plate region
    self.debug_imshow("License Plate", licensePlate)
    self.debug_imshow("ROI", roi, waitKey=True)
    break

    # return a 2-tuple of the license plate ROI and the contour
    # associated with it
    return (roi, lpCnt)

def build_tesseract_options(self, psm=7):
    # tell Tesseract to only OCR alphanumeric characters
    alphanumeric = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
    options = "-c tessedit_char_whitelist={}".format(alphanumeric)
    # set the PSM mode
    options += " --psm {}".format(psm)
    # return the built options string
    return options

def find_and_ocr(self, image, psm=7, clearBorder=False):
    # initialize the license plate text
    lpText = None
    # convert the input image to grayscale, locate all candidate
    # license plate regions in the image, and then process the
    # candidates, leaving us with the 'actual' license plate
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    candidates = self.locate_license_plate_candidates(gray)
    (lp, lpCnt) = self.locate_license_plate(gray, candidates,
                                          clearBorder=clearBorder)
    # only OCR the license plate if the license plate ROI is not
    # empty
    if lp is not None:
        # OCR the license plate
        options = self.build_tesseract_options(psm=psm)
        pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files (x86)\Tesseract-OCR\tesseract.exe'
        lpText = pytesseract.image_to_string(lp, config=options)
        self.debug_imshow("License Plate", lp)
    # return a 2-tuple of the OCR'd license plate text along with
    # the contour associated with the license plate region
    return (lpText, lpCnt)

Activate Windows
Go to Settings to activate Windows.
Ln: 68 Col: 0
```

```
test.py - C:\Users\Admin\Desktop\Final_Project\test.py (3.6.3)
File Edit Format Run Options Window Help

from apnr import ANPR
from imutils import paths
import argparse
import imutils
import cv2

def cleanup_text(text):
    # strip out non-ASCII text so we can draw the text on the image
    # using OpenCV
    return "".join([c if ord(c) < 128 else "" for c in text]).strip()

ap = argparse.ArgumentParser()
ap.add_argument("-i", "--input", default="testing_images",
                help="path to input directory of images")
ap.add_argument("-c", "--clear-border", type=int, default=-1,
                help="whether or to clear border pixels before OCR'ing")
ap.add_argument("-p", "--psm", type=int, default=7,
                help="default PSM mode for OCR'ing license plates")
ap.add_argument("-d", "--debug", type=int, default=-1,
                help="whether or not to show additional visualizations")
args = vars(ap.parse_args())

anpr = ANPR(debug=args["debug"] > 0)
# grab all image paths in the input directory
imagePaths = sorted(list(paths.list_images(args["input"])))

import serial
import time
arduino = serial.Serial(port='COM8', baudrate=9600, timeout=.1)

def write_read(x):
    #arduino.open()
    arduino.write(bytes(x))
    time.sleep(0.05)
    data = arduino.readline()
    return data

def remove(string):
    return string.replace(" ", "")

#anpr.detect_license_plate(image[0].data)

Activate Windows
Go to Settings to activate Windows.
Ln: 42 Col: 27
```

```
test.py - C:\Users\Admin\Desktop\Final_Project\test.py (3.6.3)
File Edit Format Run Options Window Help

import time
import cv2
import urllib
import requests
import anpr

def remove(string):
    return string.replace(" ", "")

imagePath='testing_images/3.jpg'

image = cv2.imread(imagePath)
#image = imutils.resize(image, width=500)

(lpText, lpCnt) = anpr.find_and_ocr(image, psm=args["psm"],
                                   clearBorder=args["clear_border"] > 0)

if lpText is not None and lpCnt is not None:
    # draw the bounding box on the license plate
    box = cv2.boxPoints(cv2.minAreaRect(lpCnt))
    box = box.astype("int")
    cv2.drawContours(image, [box], -1, (0, 255, 0), 2)

    (x, y, w, h) = cv2.boundingRect(lpCnt)
    cv2.putText(image, cleanup_text(lpText), (x, y - 15),
                cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 255, 0), 2)
    lpText=remove(lpText)
    print("[INFO] {}".format(lpText))

    url = ("http://demoproject.in/parking_iot/request5.aspx?car_no="+lpText)
    import urllib.request
    contents = urllib.request.urlopen(url).read()
    data=contents
    if data == "b'*Success=1'":
        print(data)
        num = 1
        arduino.write("1".encode())
        #time.sleep(0.05)
        #value = write_read(num)

    cv2.imshow("Output ANPR", image)
    #cv2.waitKey(0)

Activate Windows
Go to Settings to activate Windows.

No new notifications
15:08
20-05-2021
```

CHAPTER 5

RESULTS AND DISCUSSION

PROJECT SCREENSHOTS



User Login

User Name:-		
Password:-		
	Login	Sign Up



Add New User

Name:-		
Email ID:-		
Contact Number		
Address:-		
Password:-		
Car Name:-		
Car No:-		
	Submit	Cancel



3:31 PM

8.5KB/s      68



demoproject.in/parking_iot/

5



Car Parking

Home

Manage Slots

Logout

PK1		BookPK1
PK2		BookPK2
PK3		BookPK3
PK4		BookPK4

3:32 PM

0.0KB/s      68



demoproject.in/parking_iot/

5



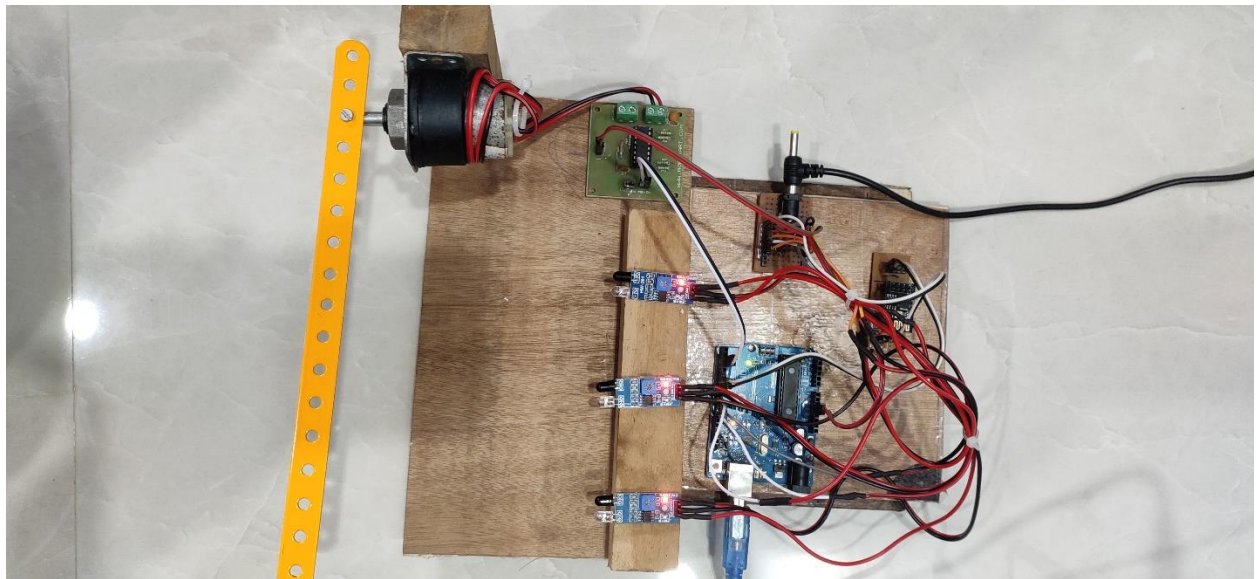
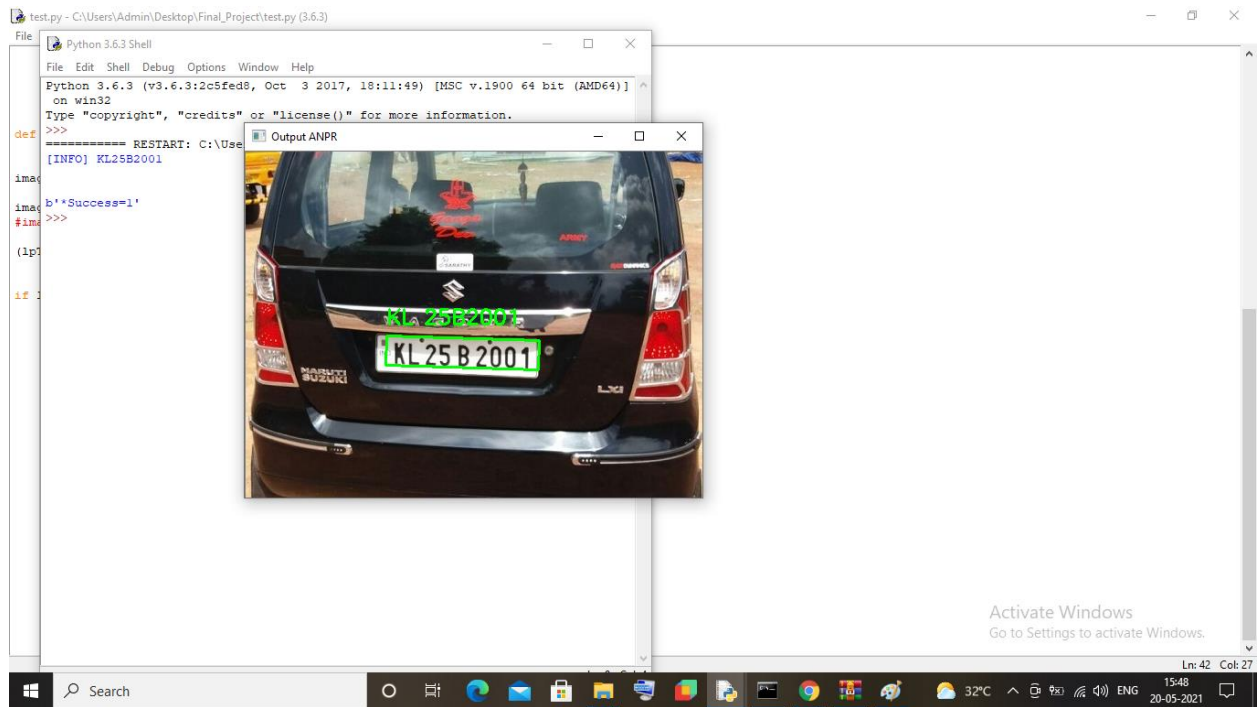
Car Parking

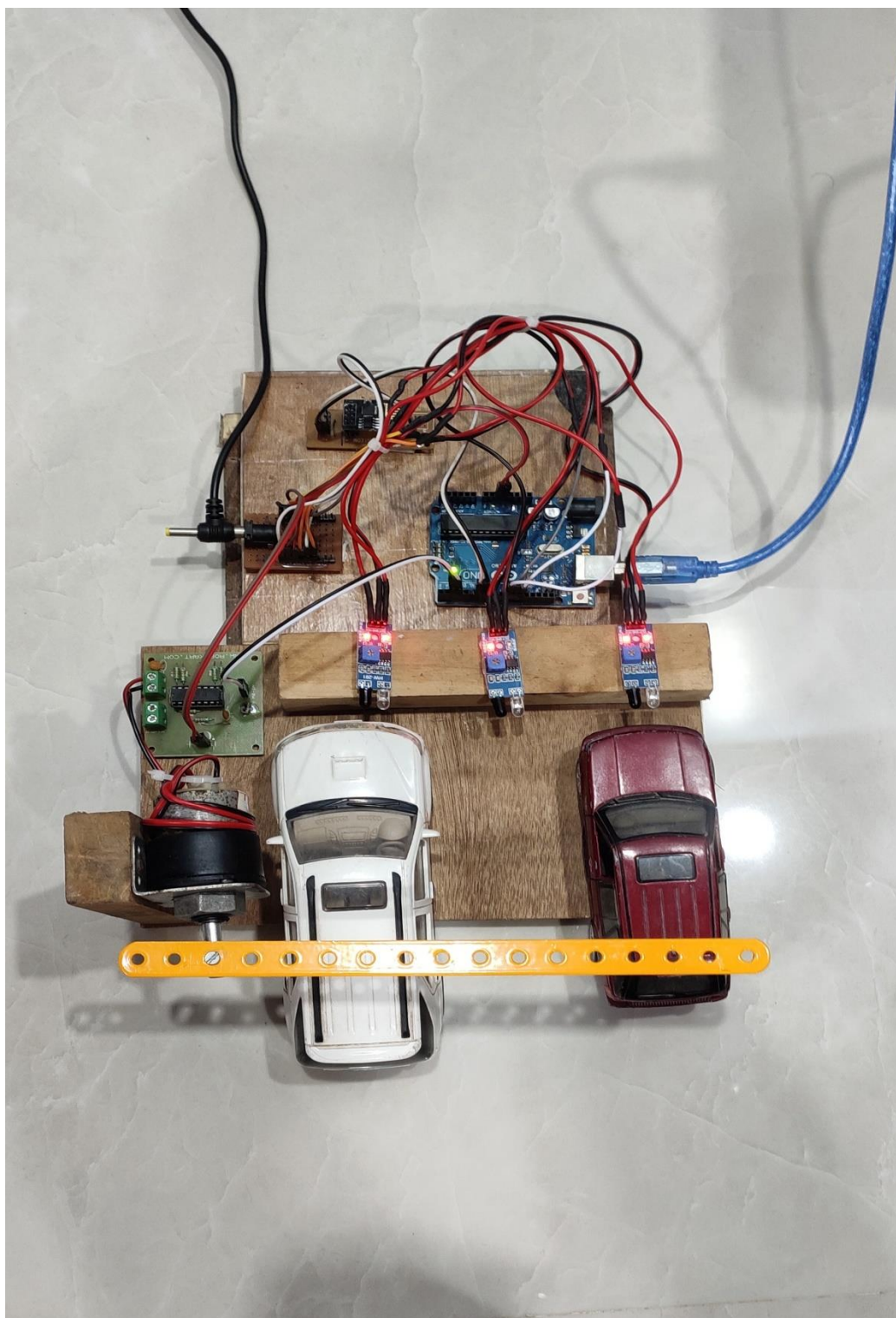
Home

Manage Slots

Logout

PK1		BookPK1
PK2		BookPK2
PK3		BookPK3
PK4		BookPK4





CHAPTER 6

CONCLUSION

CONCLUSION

This paper has proposed a vacant parking slot detection and tracking system that fuses the sensors. This paper has presented that 1) parking slot markings can be successfully detected and tracked by fusing two off-the-shelf parking aid systems, 2) parking slot markings can be reliably detected by combining sequential detection results, 3) occupancy of parking slot can be efficiently classified by treating each parking slot region as a cell of an occupancy grid, and 4) parking slot markings can be tracked robust against severe occlusions. As a result of quantitative evaluation using practical databases, the proposed system achieves 97.8% recall and 95.8% precision for parking slot marking detection, 98.1% classification rate for parking slot occupancy classification, and accuracy of 3.1 cm and 0.9 ° for parking slot marking tracking.

CHAPTER 7

FUTURE SCOPE

FUTURE SCOPE

- In our future work, we will consider the security aspects of our prototype system as well as implement the system in large scales in real environments.
- we will detail and develop this new adaptable self-organization protocol for wireless sensor networks by performing simulations in order to demonstrate its strength by comparing it with other existing self-organization protocols.
- Improve the resolution quality for larger areas for extra detailing.
- Make a user-friendly app for the users.

REFERENCES

- [1] S. Hiramatsu, A. Hibi, Y. Tanaka, T. akinami, Y. Iwata, and M. Nakamura, "Rearview camera based parking assist system with voice guidance," presented at the Proc. SAE World Congr. Exhib., Detroit, MI, USA, Apr. 2002, Paper 2002-01-0759.
- [2] M. Furutani, "Obstacle detection systems for vehicle safety," presented at the Proc. SAE Converg. Int. Congr. Expo. Transp. Electron., Detroit, MI, USA, Oct. 2004, Paper 2004-21- 0057.
- [3] Y. Kageyama, Look, No Hand! New Toyota Parks Itself, Jan. 14, 2004. [Online]. Available: <http://www.cnn.com/>
- [4] H. G. Jung, C. G. Choi, P. J. Yoon, and J. Kim, "Novel user interface for semi-automatic parking assistance system," in Proc. 31st FISITA World Autom. Congr., Oct. 2006, pp. 1–10.
- [5] H. Satonaka, M. Okuda, S. Hayasaka, T. Endo, Y. Tanaka, and T. Yoshida, "Development of parking space detection using an ultrasonic ensor," in Proc. 13th World Congr. Intell. Transp. Syst. Serv., Oct. 2006, pp. 1–10.
- [6] P. Degerman, J. Pohl, and M. Sethson, "Hough transform for parking space estimation using long range ultrasonic sensors," presented at the Proc. SAE World Congr. Exhib., Detroit, MI, USA, pr. 2006, Paper 2006-01-0810.
- [7] W. J. Park, B. S. Kim, D. E. Seo, D. S. Kim, and K. H. Lee, "Parking space detection using ultrasonic sensor in parking assistance system," in Proc. IEEE Intell. Veh. Symp., Jun. 2008, pp. 1039–1044.
- [8] S. H. Jeong, C. G. Choi, J. N. Oh, P. J. Yoon, B. S. Kim, M. Kim, and K. H. Lee, "Low cost design of parallel parking assist system based on an ultrasonic sensor," Int. J. Autom. Technol., vol. 11, no. 3, pp. 409–416, Jun. 2010.
- [9] Ford 2013 TAURUS. [Accessed: Mar. 2013]. [Online]. Available: [http://www.ford.com/cars/taurus/features/#page=Fea ture18](http://www.ford.com/cars/taurus/features/#page=Fea%20ture18).